

Definition: Lean Transpiler Layer

Lean Transpiler Layer - Lean $\rightleftarrows$ Graph DSL $\rightleftarrows$ IR Tokens

This page defines the **Lean transpiler layer**, the component that connects Lean surface syntax to the universal graph-token IR. It provides a reversible pipeline:

Lean $\rightarrow$ Graph DSL $\rightarrow$ IR tokens $\rightarrow$ Graph DSL $\rightarrow$ Lean

This is **Layer 3** of the architecture, sitting above the DSL types and the IR encoder/decoder.

1. DSL Construction Helpers

These helpers simplify building Graph structures from Lean values.

```
namespace GraphDSL

def mkVar (name : String) (pol : Polarity := Polarity.zero) : Entity :=
  { class := "E", polarity := pol, id := EntityId.var name, chain := "*" }

def mkNum (n : Int) (pol : Polarity := Polarity.zero) : Entity :=
  { class := "E", polarity := pol, id := EntityId.num n, chain := "*" }

def mkEqn (pol : Polarity := Polarity.zero) : Entity :=
  { class := "E", polarity := pol, id := EntityId.eqn, chain := "*" }

def mkAttr (target : Entity) (key value : String) (pol : Polarity := Polarity.zero) : Attribute :=
  { class := "A", polarity := pol, target := target.id, key := key, value := value, chain := "*" }

def mkRelation (pol : Polarity) (op : RelationOp)
  (e1 e2 e3 : Entity) : Relation :=
  { class := "R", polarity := pol, op := op, e1 := e1, e2 := e2, e3 := e3, chain := "*" }

def mkOperation (pol : Polarity) (op : OperationOp)
  (e1 e2 : Entity) : Operation :=
  { class := "O", polarity := pol, op := op, e1 := e1, e2 := e2, chain := "*" }

end GraphDSL
```

2. Example: Lean $\rightarrow$ Graph DSL

A simple Lean structure:

```
structure SimpleEq :=  
  (x : String)  
  (a : Int)  
  (b : Int)  
  (rhs : Int)
```

Transpiling to a Graph:

```
open GraphDSL  
  
def transpileSimpleEqToGraph (e : SimpleEq) : Graph :=  
  let x := mkVar e.x  
  let n3 := mkNum e.a  
  let n5 := mkNum e.b  
  let n12 := mkNum e.rhs  
  let t1 := mkVar "T1"  
  let t2 := mkVar "T2"  
  let eqn := mkEqn  
  
  let aN3 := mkAttr n3 "val" (toString e.a)  
  let aN5 := mkAttr n5 "val" (toString e.b)  
  let aN12 := mkAttr n12 "val" (toString e.rhs)  
  let aX := mkAttr x "type" "unknown"  
  
  let rMul := mkRelation Polarity.pos RelationOp.mul x n3 t1  
  let rAdd := mkRelation Polarity.pos RelationOp.add t1 n5 t2  
  let rEq := mkRelation Polarity.zero RelationOp.eq t2 n12 eqn  
  
  let oSub := mkOperation Polarity.neg OperationOp.sub n5 eqn  
  let oDiv := mkOperation Polarity.neg OperationOp.div n3 eqn  
  let oIso := mkOperation Polarity.zero OperationOp.iso x eqn  
  
  {  
    entities := [x, n3, n5, n12, t1, t2, eqn],  
    attributes := [aN3, aN5, aN12, aX],  
    relations := [rMul, rAdd, rEq],  
    operations := [oSub, oDiv, oIso]  
  }
```

3. Lean $\rightarrow$ IR Tokens

```
def transpileSimpleEqToTokens (e : SimpleEq) : List String :=  
  encodeGraph (transpileSimpleEqToGraph e)
```

4. IR Tokens → Graph DSL → Lean

Reconstructing Lean values from a Graph:

```
def graphToSimpleEq (g : Graph) : Option SimpleEq :=
  let xs := g.entities.filter (fun e => match e.id with | EntityId.var _ => true | _ => false)
  let nums := g.entities.filter (fun e => match e.id with | EntityId.num _ => true | _ => false)
  match xs, nums with
  | x :: _, n3 :: n5 :: n12 :: _ =>
    let getVal (e : Entity) : Int :=
      match g.attributes.find? (fun a => a.target = e.id ∧ a.key = "val") with
      | some a => a.value.toInt!
      | none => 0
    some {
      x := match x.id with | EntityId.var s => s | _ => "x",
      a := getVal n3,
      b := getVal n5,
      rhs := getVal n12
    }
  | _, _ => none
```

Full round-trip:

```
def tokensToSimpleEq (ts : List String) : Option SimpleEq :=
  graphToSimpleEq (decodeGraph ts)
```

5. Transpiler Abstraction

A reusable structure for any Lean domain type:

```
structure Transpiler (α : Type) :=
  (toGraph : α → Graph)
  (toTokens : α → List String)
  (fromGraph : Graph → Option α)
  (fromTokens : List String → Option α)
```

This allows domain-specific transpilers for:

- algebraic equations
- systems of equations
- calculus expressions
- linear algebra
- logic and sets
- probability

All using the same universal graph-token IR.

Purpose of This Page

This page defines the **Lean transpiler layer**, enabling reversible translation between Lean surface syntax, the Graph DSL, and the universal graph-token IR. It completes the Lean  IR pipeline and prepares the system for SLM integration and VS Code autocompletion.